Experiment-9

Aim: Demonstrate the Bankers Algorithm for Deadlock Avoidance.

Description

The **Banker's Algorithm** is a classic resource allocation and deadlock avoidance algorithm developed by Edsger Dijkstra. It is designed for systems with multiple instances of resources. The algorithm gets its name from the banking system, where a banker ensures that the bank never runs out of cash to satisfy the maximum cash requirements of its clients.

Algorithm

I. Initialization & Input Step 1: Start. Step 2: Read the total number of processes (n) and total number of resource types (m). Step 3: Read the Allocation matrix of size n x m. Step 4: Read the Max matrix of size n x m. Step 5: Read the Available resources array of size m.

II. Need Matrix Calculation Step 1: Loop through each process i (from 0 to n-1). Step 2: Loop through each resource j (from 0 to m-1). Step 3: Calculate the required resources using the formula: Need[i][j] = Max[i][j] - Allocation[i][j].

III. Safety Algorithm Execution Step 1: Initialize an array finish[n] to 0 (indicating no process is finished yet). Step 2: Initialize a count variable to 0 (to track the number of processes successfully added to the safe sequence). Step 3: While count < n, perform the following:

1. Set a flag found = 0.
2. Loop through all processes i (from 0 to n-1):
    - If finish[i] == 0 (process is not yet finished):
        - Check if Need[i][j] <= Available[j] for all resources j.
        - If the condition is satisfied (process can be executed):
            - Add its allocated resources back to the available pool: Available[j] += Allocation[i][j].
            - Add process i to the safe sequence array.
            - Increment count by 1.
            - Mark the process as finished: finish[i] = 1.
            - Set found = 1 (indicating at least one process was processed in this pass).
3. If found == 0 (no process could be executed in this entire pass), break out of the loop (system is deadlocked/unsafe).

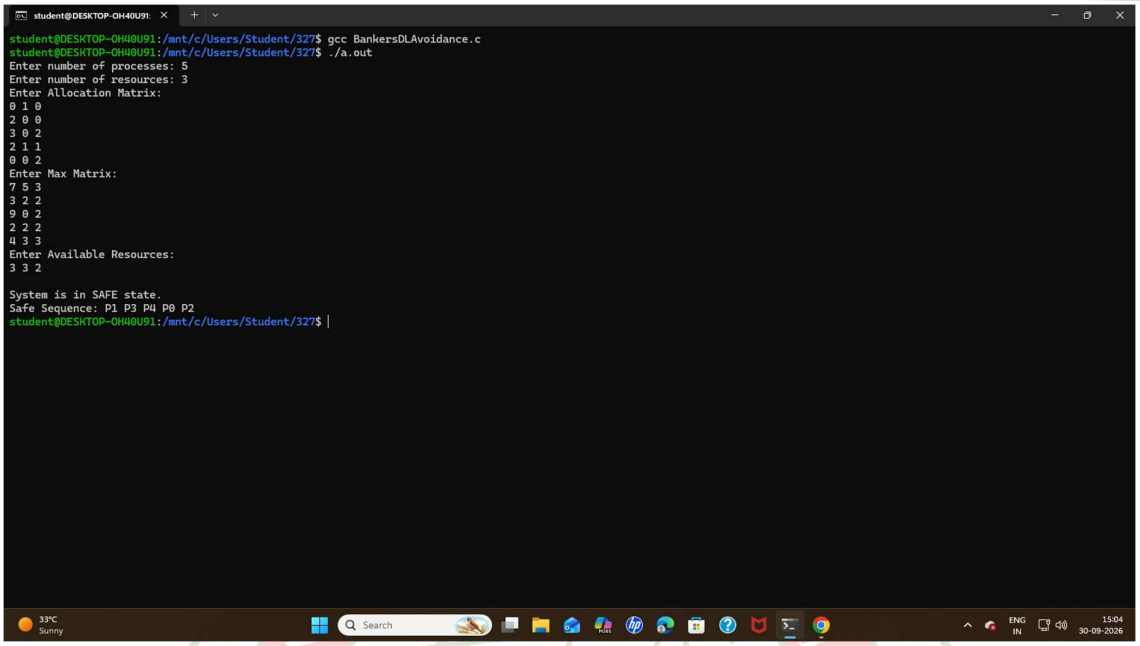
IV. Final State Check & Output Step 1: Check if count == n (all processes successfully finished). Step 2: If true, print "System is in SAFE state" and display the generated safe sequence. Step 3: If false, print "System is NOT in a SAFE state". Step 4: Stop.

Code:

```
#include <stdio.h>
int main(){
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);
    int allocation[n][m], max[n][m], need[n][m];
    int available[m], finish[n], safe[n];
    printf("Enter Allocation Matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &allocation[i][j]);
    printf("Enter Max Matrix:\n");
    for (int i = 0; i < n; i++)
```

```
            for (int j = 0; j < m; j++)
                scanf("%d", &max[i][j]);
        printf("Enter Available Resources:\n");
        for (int j = 0; j < m; j++)
            scanf("%d", &available[j]);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++)
                need[i][j] = max[i][j] - allocation[i][j];
        for (int i = 0; i < n; i++)
            finish[i] = 0;
        int count = 0;
        while (count < n){
            int found = 0;
            for (int i = 0; i < n; i++){
                if (finish[i] == 0){
                    int possible = 1;
                    for (int j = 0; j < m; j++){
                        if (need[i][j] > available[j]){
                            possible = 0;
                            break;
                        }
                    }
                    if (possible){
                        for (int j = 0; j < m; j++)
                            available[j] += allocation[i][j];
                        safe[count] = i;
                        count++;
                        finish[i] = 1;
                        found = 1;
                    }
                }
            }
            if (found == 0)
                break;
        }
        if (count == n){
            printf("\nSystem is in SAFE state.\n");
            printf("Safe Sequence: ");
            for (int i = 0; i < n; i++)
                printf("P%d ", safe[i]);
            printf("\n");
        }
        else{
            printf("\nSystem is NOT in a SAFE state.\n");
        }
        return 0;
    }
```

Output:



Aim: Demonstrate the Bankers Algorithm for Deadlock Detection.

Description

The **Banker's Algorithm** is primarily known as a deadlock avoidance method, but it can also be adapted for **deadlock detection** in systems with **multiple instances of each resource type**. While avoidance relies on a process's *maximum potential need*, detection analyzes the *actual, current requests* of processes to determine if a deadlock already exists or is unavoidable based on current demands

Algorithm

I. Initialization & Input Step 1: Start. Step 2: Read the total number of processes (n) and total number of resource types (m). Step 3: Read the Available resources array of size m. Step 4: Read the Allocation matrix of size n x m. Step 5: Read the Request matrix of size n x m.

II. Setup Work and Finish Arrays Step 1: Initialize a work array of size m. Set work[j] = available[j] for all resources j. Step 2: Initialize a finish array of size n. Step 3: For each process i (from 0 to n-1):

1. Check if the process has zero resources allocated (allocation[i][j] == 0 for all j).
2. If it has no allocations, set finish[i] = 1.
3. Otherwise, set finish[i] = 0.

III. Detection Algorithm Execution Step 1: Enter a continuous loop. Step 2: Inside the loop, set a flag found = 0 to track if any process is resolved in the current pass. Step 3: Loop through all processes i (from 0 to n-1):

1. If finish[i] == 0 (process is currently unresolved):
    - Check if request[i][j] <= work[j] for all resources j.
    - If the condition is satisfied (requests can be fulfilled):
        - Add its allocated resources back to the work pool: work[j] += allocation[i][j].
        - Mark the process as resolved: finish[i] = 1.
        - Set found = 1.
        - Print that process i can finish. Step 4: If found == 0 (no processes could be resolved in this entire pass), break out of the continuous loop.

IV. Final State Check & Output Step 1: Set a flag deadlock = 0. Step 2: Loop through all processes i. If any finish[i] == 0, set deadlock = 1 and break the loop. Step 3: If deadlock == 1:

1. Print "Deadlock detected."
2. Loop through and print the IDs of all processes where finish[i] == 0 (the deadlocked processes). Step 4: If deadlock == 0, print "No deadlock detected." Step 5: Stop.

Code:

```
#include <stdio.h>
int main(){
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);
    int available[m];
    int allocation[n][m];
    int request[n][m];
    int work[m];
    int finish[n];
    printf("\nEnter Available resources:\n");
    for (int j = 0; j < m; j++)
        scanf("%d", &available[j]);
    printf("\nEnter Allocation Matrix:\n");
    for (int i = 0; i < n; i++){
        for (int j = 0; j < m; j++)
            scanf("%d", &allocation[i][j]);
    }
    printf("\nEnter Request Matrix:\n");
    for (int i = 0; i < n; i++){
        for (int j = 0; j < m; j++)
            scanf("%d", &request[i][j]);
    }
    for (int j = 0; j < m; j++)
        work[j] = available[j];
    for (int i = 0; i < n; i++){
        finish[i] = 0;
        int zero = 1;
        for (int j = 0; j < m; j++){
            if (allocation[i][j] != 0){
                zero = 0;
                break;
            }
        }
        if (zero == 1)
            finish[i] = 1;
    }
    while (1){
        int found = 0;
        for (int i = 0; i < n; i++){
            if (finish[i] == 0){
                int possible = 1;
                for (int j = 0; j < m; j++){
                    if (request[i][j] > work[j]){
                        possible = 0;
                        break;
```

```
                    }
                }
                if (possible == 1){
                    for (int j = 0; j < m; j++)
                        work[j] += allocation[i][j];
                    finish[i] = 1;
                    found = 1;
                    printf("P%d can finish.\n", i);
                }
            }
        }
        if (found == 0)
            break;
    }
    int deadlock = 0;
    for (int i = 0; i < n; i++){
        if (finish[i] == 0){
            deadlock = 1;
            break;
        }
    }
    if (deadlock == 1){
        printf("\nDeadlock detected.\n");
        printf("Deadlocked processes: ");
        for (int i = 0; i < n; i++){
            if (finish[i] == 0)
                printf("P%d ", i);
        }
        printf("\n");
    }
    else{
        printf("\nNo deadlock detected.\n");
    }
    return 0;
}
```

Output:

